

Agile Software Development

14-Aug-26

vijaynathani.github.io

1

During Introduction:

- Name, Number of years in Software Development, Role now.
- Problems faced on this project or previous. Make a list. At the end, most problems should be solved.
- How familiar are you with Scrum? 10 == read both books; I know what it's about; I've practiced it! 1 == seemed kinda interesting so I decided to take a training course
- What development model best describes your project working environment? 10 == consulting: firm is working with external clients; client driven development. 1 == corporate: managing development of in-house product or service

Course Content

<https://shorturl.at/XK5fa>

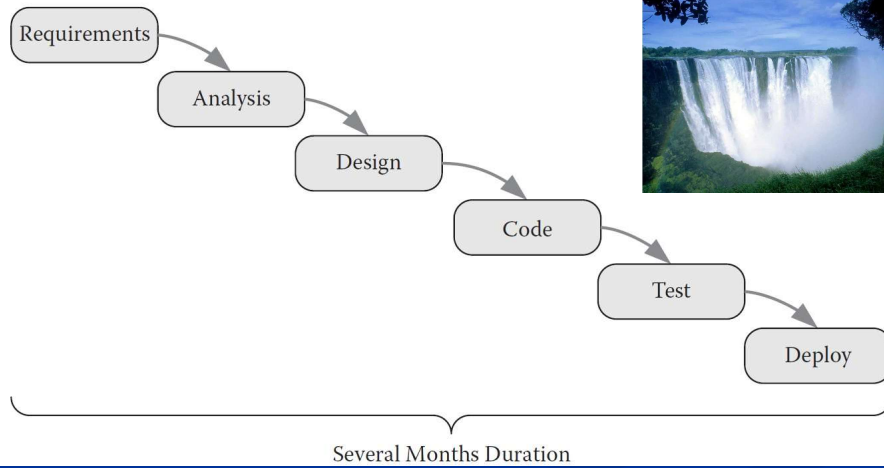
History

Listening, Designing, Coding, Testing –
That's all there is to software. Anyone, who
tells you different, is selling something

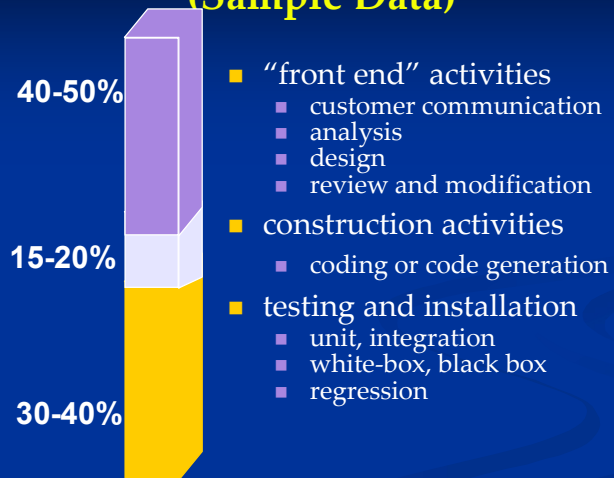
Predictive Processes

He, who chooses the beginning of a road,
chooses the place it leads to.

The Waterfall Model



Effort Allocation (Sample Data)

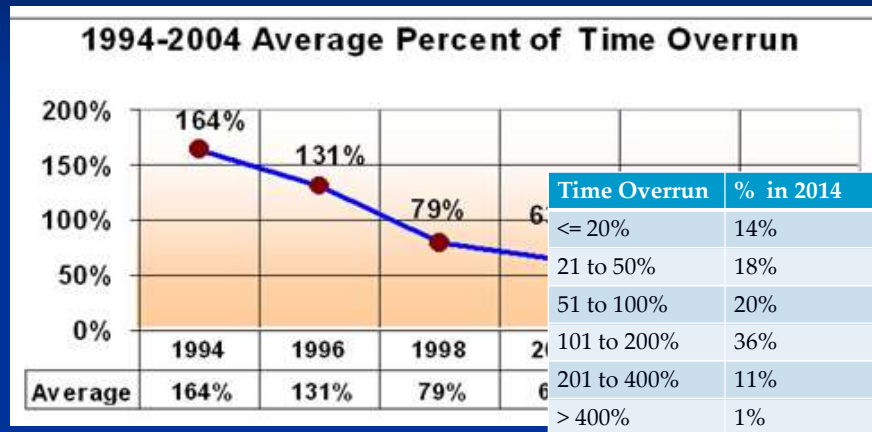


Cost Overruns over Original Estimate

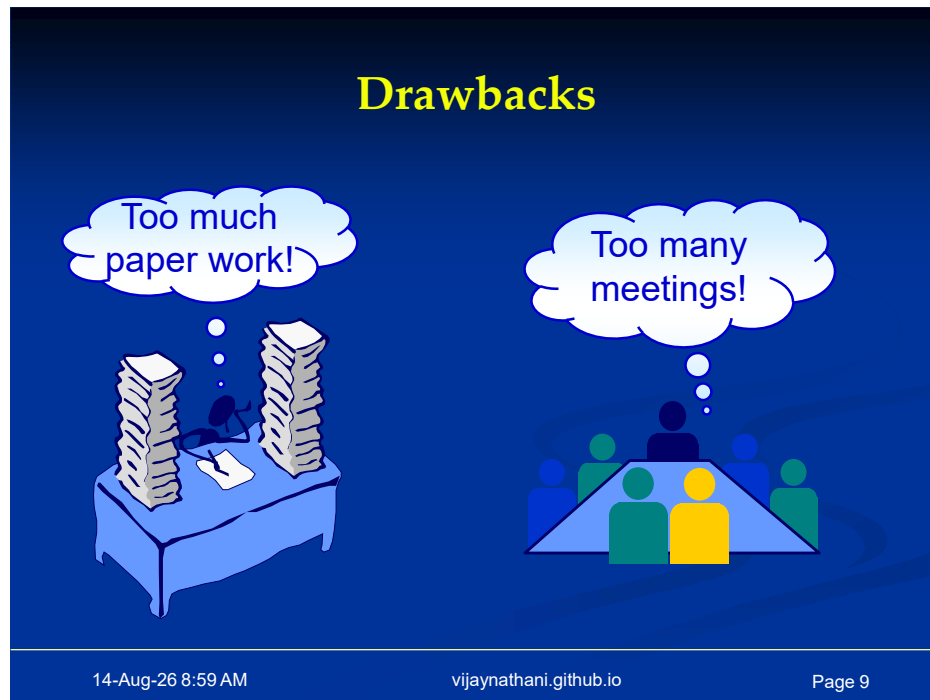


Cost overruns table as per 2014 CHAOS report from Standish group

Time Overruns over Original Estimate



Time overruns table as per 2014 CHAOS report from Standish group



Studies of documentation produced during conventional software development indicated that for an average 1,000-function point software project (about 125,000 lines of code):

- Requirements documents average 300 pages
- Plans average 100 pages
- Design documents average more than 1,500 pages
- User manuals average more than 600 pages
- Test reports average more than 5,000 pages

The mere presence of a detailed specification may act to the detriment of cooperation between the parties, encouraging both parties to hide behind the specification rather than seeking mutual beneficial solutions

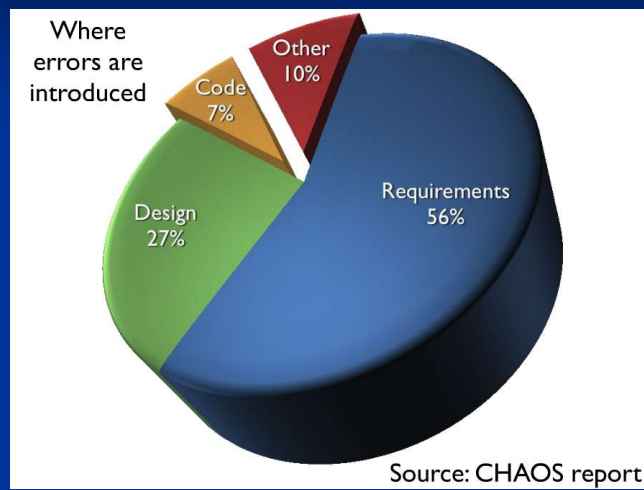
Example of paperwork:

Excessive documentation does not add value but only eats up resources and time.

Unclear Requirements



Source of Errors



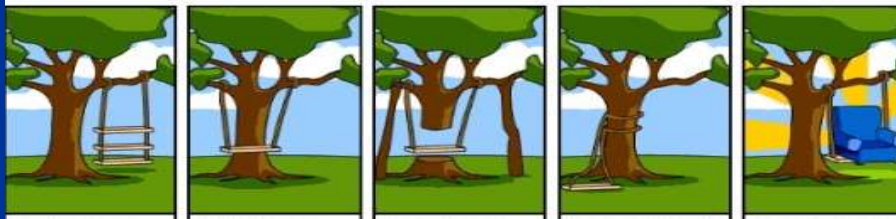
See: <http://www.infoq.com/presentations/hellesoy-bdd-rspec>

Software Construction ...

How the
customer
explained it

How the
Analyst
designed it

How the Business
Analyst described
it



How the Project
Leader
understood it

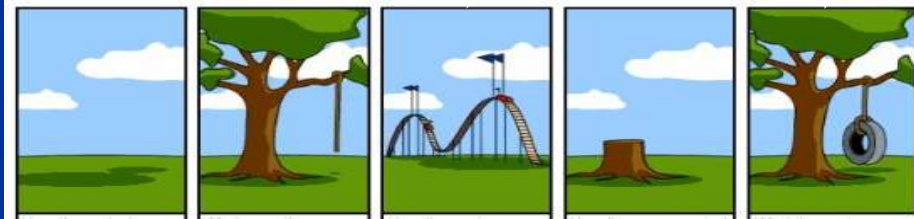
How the
Programmer
wrote it

Software Construction ...

How the
Project was
documented

How the
customer
was billed

What the
customer really
needed



How the
Operations
installed

How it was
supported

Legal Implications of the Current Situation

- 2002 survey of information technology organizations
 - 78% have been involved in disputes that ended in litigation

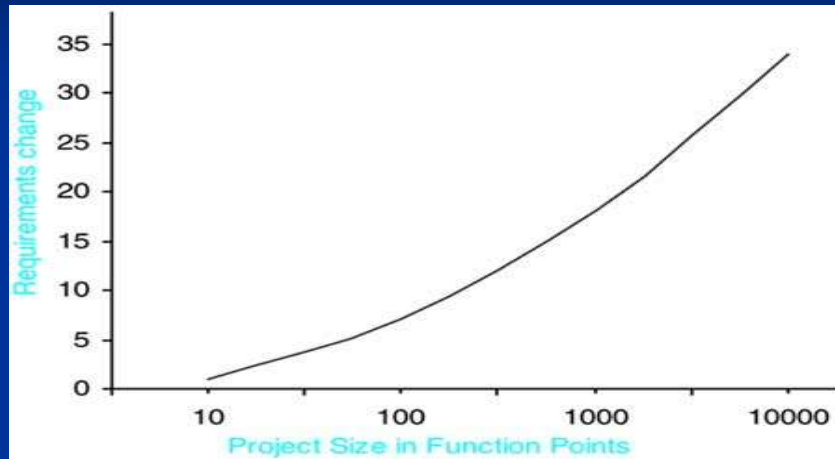
For the organizations that entered into litigation:

In 67% of the disputes, the functionality of the information system as delivered did not meet not up to the claims of the developers

In 56% of the disputes, the promised delivery date slipped several times

In 45% of the disputes, the defects were so severe that the information system was unusable

How to handle change?



14-Aug-26 8:59 AM

[vijaynathani.github.io](https://github.com/vijaynathani)

Page 15

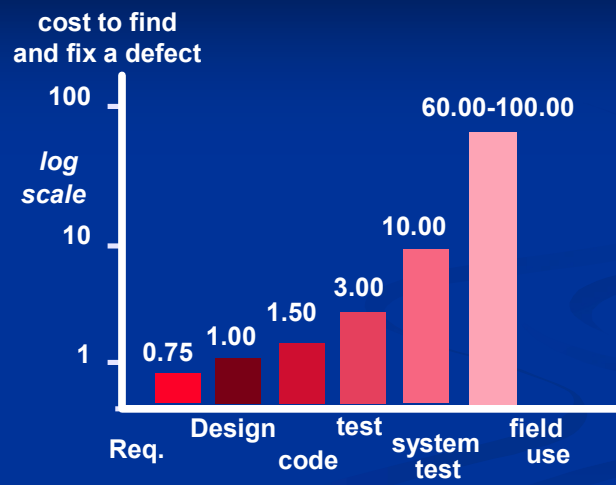
Software development is about change:

- Changing requirements
- Change in technology
- Change in people involved
- Change in competition, customer, government rules

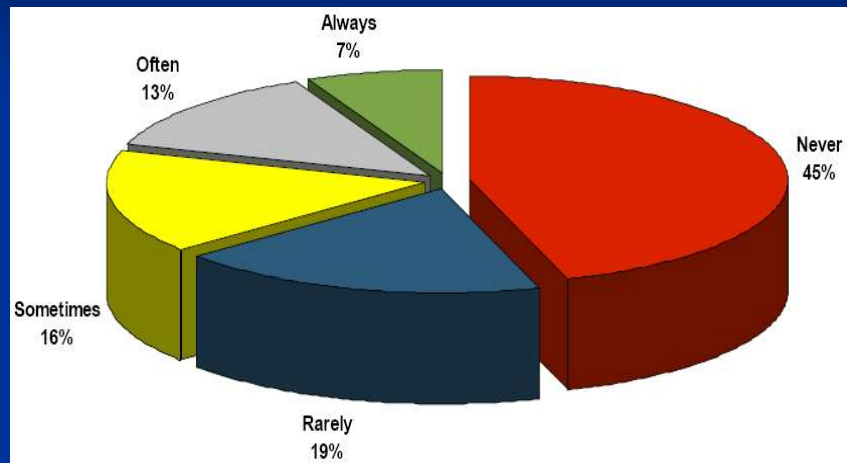
Planned development is like planned economy.

As per 2014 CHAOS report from Standish group. On Small projects, 24% features are due to change requests in end product; 35% for medium projects and 39% for large projects.

Cost of Change (COC) rises Exponentially



YAGNI violated



Standish group study reported at XP2002 by Jim Johnson.

Truth

- Customers are afraid
- Developers are afraid
- Management is afraid



14-Aug-26 8:59 AM

[vijaynathani.github.io](https://github.com/vijaynathani)

Page 18

Customers are afraid

- They won't get what they asked for.
- They'll ask for the wrong thing.
- They'll pay too much for too little.
- They must surrender control of their career to techies, who don't care.
- They won't ever see a meaningful plan.
- The plans they do see will be fairy-tales.
- They won't know what's going on.
- They'll be held to their first decisions and won't be able to react to changes in the business.
- No one will tell them the truth.

Developers are afraid

- They will be told to do more than they know how to do.
- They will be told to do things that don't make sense.
- They will be given responsibility without authority.
- They won't be given clear definitions of what needs to be done.
- That they'll have to sacrifice quality for deadlines.
- That they'll have to solve hard problems without help.

- That they won't have enough time to succeed.
- The project will produce the wrong product

Tan Lines From Typical Summer Activities



Source: <http://www.kerrolisa.com/115637.jpg>

The Problem

“At the end of a waterfall project, the client says, maybe that’s what I said I needed, but that’s not what I need now”

– Robert Martin, CEO, Object Mentor

Both the developer and the customer are battered and bloody, if not dead, after the typical project.

And the project delivers sub-par software at best. Usually it’s junk that everyone is at least a little bit ashamed of, in private.

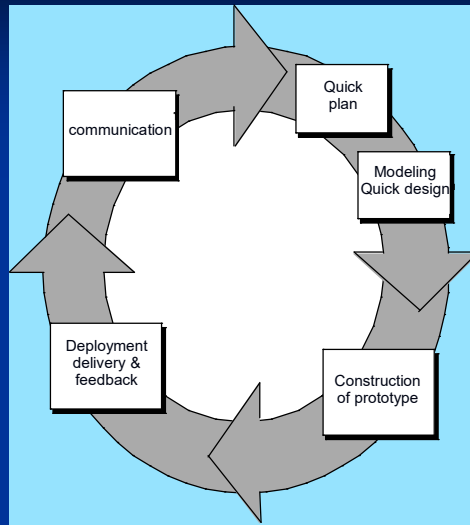
The software industry is in a sorry state, at least from the standpoint of the people who are part of it.

Developers hate life.

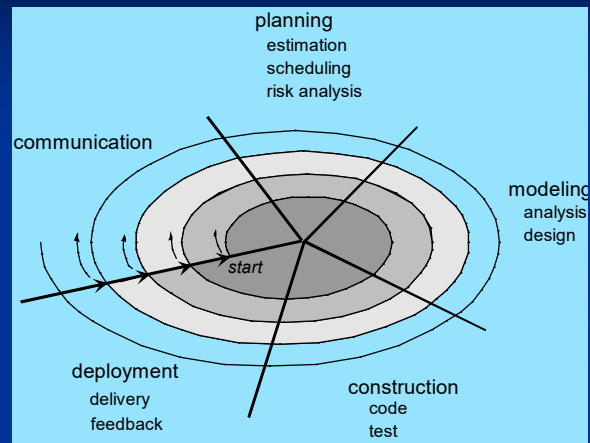
Customers rarely like what they get.

The software stinks.

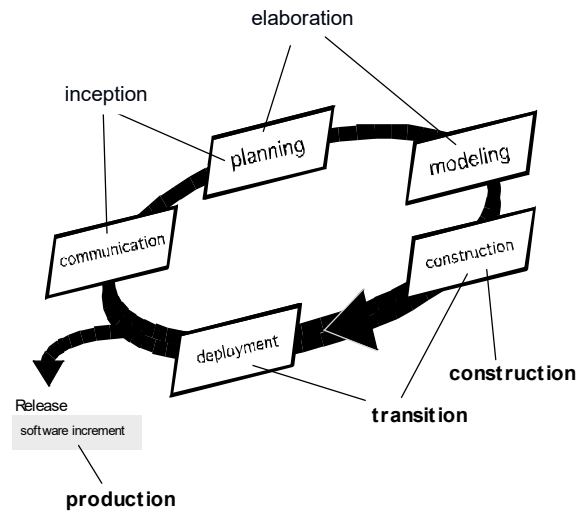
Evolutionary Models: Prototyping



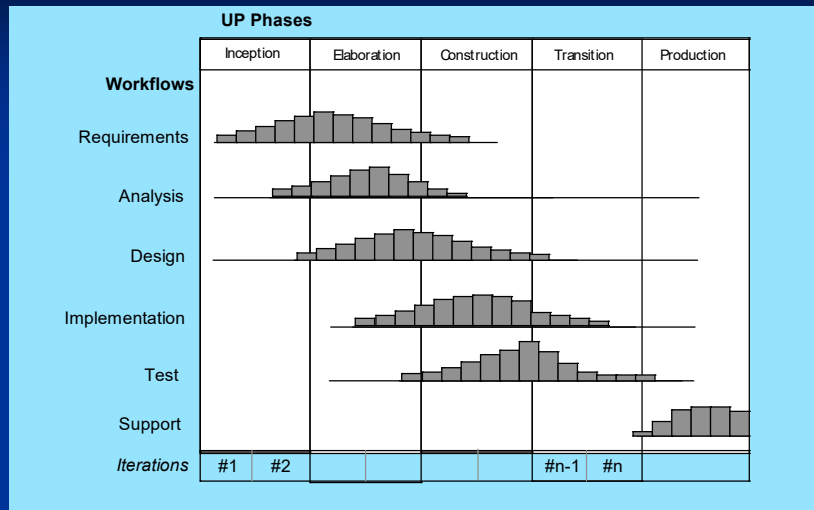
Evolutionary Models: The Spiral



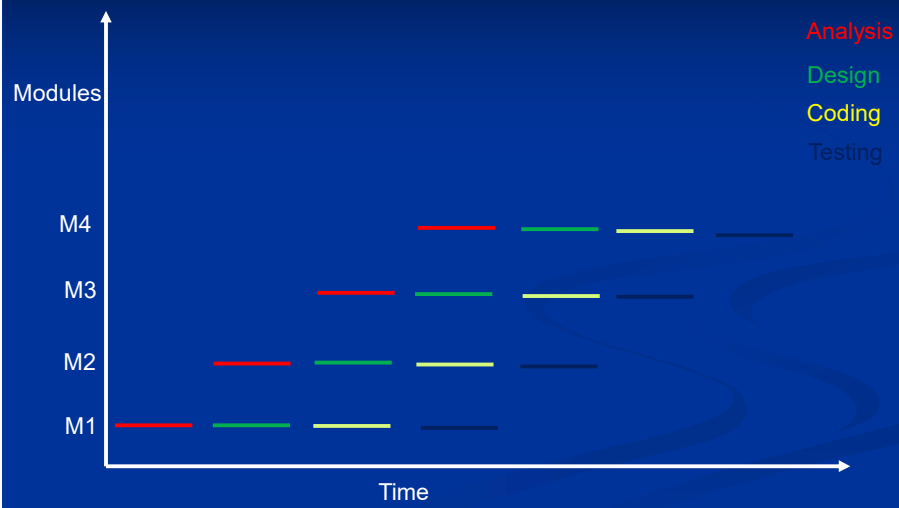
The Unified Process (UP)



UP Phases



Unified Process



Any other Process?

- Code & Fix
 - Cowboy Programming
- What is your process?
 - What is your definition of a Process?

14-Aug-26 8:59 AM

vijaynathani.github.io

Page 26

Definition of Process: What you do under pressure is your process, everything else is window dressing or political correctness.

All methods are someone else's idea about what you should do when you develop software. It may be useful, from time to time, to borrow from those ideas and integrate them into your own style. It is essential, however, to transcend any method, even your own idiosyncratic method, and "just do it."

Software development is like riding a surfboard—there is no process that will assure a successful ride, nor is there any process that will assure that you will interact propitiously with the other surfers sharing the same wave. Published processes, like published methods, provide observational data from which you can learn and thereby improve your innate abilities—just as observation of master surfers enables you to improve yourself.

=====

Methodology is a social construction. Your "methodology" is everything you regularly do to get your software out. It includes who you hire, what you hire them for, how they work together, what they produce, and how they share. It is the combined job descriptions, procedures, and conventions of everyone on your team. It is the product of your particular ecosystem and is therefore a unique construction of your organization.

All organizations have a methodology: It is simply how they do business. Even the proverbial trio in a garage has a way of working away of trading information, of separating work, of putting it back together all founded on assumed values and

cultural norms. The way of working includes what people choose to spend their time on, how they choose to communicate, and how decision-making power is distributed.